

```

#include<iostream>
using namespace std;

class Employee {
    int id;
    int salary;
public:
    Employee(int a = 0, int b = 0) {
        id = a;
        salary = b;
    }
    void Print() { cout << "ID= " << id << "\tSalary= " << salary << endl; }
};

class Node {
    Employee data;
    Node* Ptr;
public:
    Node() { data = NULL; Ptr = NULL; } //constructor..
    Node(const Employee& d, Node* p) { data = d; Ptr = p; } // argument
    Construcotr..
    void setData(Employee d) { data = d; }
    void setNext(Node* next) { Ptr = next; }
    Employee getData() { return data; }
    Node* getNext() { return Ptr; }
};

class MyList {
    Node* fptr;
    Node* lptr;
    //Default constructor...
public:
    MyList() { fptr = lptr = NULL; }
    void insertAtFront(const Employee&);
    void insertAtBack(const Employee&);
    bool IsEmpty()const;
    void Print() ;
};

void MyList::insertAtFront(const Employee& val)
{
    Node* ptr = new Node(val, fptr);
    if (IsEmpty()) fptr = lptr = ptr; /// 1
    else fptr = ptr;
}

void MyList::insertAtBack(const Employee& val)
{
    Node* ptr = new Node(val, NULL);
    if (IsEmpty()) fptr = lptr = ptr;
    else lptr->setNext(ptr);
    lptr = ptr;
}

bool MyList::IsEmpty() const
{
    return fptr == NULL;
}

void MyList::Print()
{
    if (IsEmpty()) {
        cout << "The list is empty\n\n";
    }
}

```

[illegible]